DBMS & SQL Interview Questions

Complete Preparation Guide for Data Science Roles

76 interview questions with simple explanations, MySQL examples, business scenarios, and performance concepts

Designed for candidates preparing for Data Scientist / Data Analyst interviews at approximately the 2-year experience level.

Topics: DBMS fundamentals, keys and normalization, SQL commands, joins, subqueries, CTEs, window functions, coding problems, analytics, feature engineering, query optimization, transactions, and mock-interview guidance.

SQL dialect used in examples: MySQL 8.0. Examples use illustrative data; adjust definitions and assumptions to match the business question.

How to use this guide

Read the answer, cover the SQL query, and try to write it yourself in MySQL Workbench. Then explain the result and discuss edge cases such as NULL values, duplicates, ties, missing dates, and empty tables. An interviewer values correct reasoning as much as syntax.

Sample tables used in several examples:

employees	employee_id	name	department	salary	manager_id
Sample rows	1	Amit	IT	60000	NULL
Sample rows	2	Neha	IT	75000	1
Sample rows	3	Ravi	HR	45000	NULL
Sample rows	4	Priya	HR	55000	3
Sample rows	5	Karan	IT	75000	1
Sample rows	6	Simran	Sales	40000	NULL

customers	customer_id	customer_name	city
Sample rows	101	Aman	Bhopal
Sample rows	102	Priya	Delhi
Sample rows	103	Rahul	Bhopal
Sample rows	104	Neha	Jaipur

orders	order_id	customer_id	amount	order_date
Sample rows	1001	101	2000	2026-01-05
Sample rows	1002	102	3500	2026-01-06
Sample rows	1003	101	1500	2026-02-01
Sample rows	1004	103	5000	2026-02-10
Sample rows	1005	101	1000	2026-02-10

Part 1 — DBMS fundamentals

Q1. What is a database? What is a DBMS?

A database is an organized collection of data. A DBMS (Database Management System) is software used to create, store, retrieve, update, and manage that data. Examples include MySQL, PostgreSQL, Oracle Database, and SQL Server. An RDBMS stores data in related tables and supports relationships and constraints.

Q2. What is SQL? What are its main command categories?

SQL (Structured Query Language) is used to query and manage relational databases. DDL defines structures; DML changes data; DQL retrieves data; DCL manages permissions; TCL manages transactions. Some textbooks classify SELECT under DML rather than DQL.

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  name VARCHAR(100),
  salary DECIMAL(10,2)
);
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

Q3. What is a primary key?

A primary key uniquely identifies each row. It cannot contain NULL, and a table has one primary-key constraint, although that key can contain multiple columns.

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  name VARCHAR(100),
  salary DECIMAL(10,2)
);
```

Q4. Explain primary, foreign, candidate, composite, and unique keys.

Primary key: chosen unique identifier. Foreign key: references a key in another table or the same table. Candidate key: a minimal set of columns that uniquely identifies a row. Composite key: contains multiple columns. Unique constraint: prevents duplicate key values; NULL behavior varies by database.

```
CREATE TABLE enrollments (
  student_id INT,
  course_id INT,
  enrollment_date DATE,
  PRIMARY KEY (student_id, course_id)
);
```

Q5. What is normalization? Explain 1NF, 2NF, and 3NF.

Normalization organizes data to reduce duplication and update anomalies. 1NF means each field contains an atomic value, not a list. 2NF means 1NF plus every non-key attribute depends on the whole candidate key, not only part of a composite key. 3NF means 2NF plus no transitive dependency of non-key attributes on a candidate key. Analytics systems may intentionally denormalize data for reporting speed.

Q6. Difference between DELETE, TRUNCATE, and DROP.

DELETE removes selected rows (or all rows) and supports WHERE. TRUNCATE removes all rows but not the table structure. DROP removes the table and its structure. Transaction and rollback behavior depends on the database and execution context; be careful with destructive commands.

```
DELETE FROM employees WHERE employee_id = 10;
TRUNCATE TABLE employees;
DROP TABLE employees;
```

Q7. Difference between WHERE and HAVING.

WHERE filters rows before grouping. HAVING filters groups after aggregation.

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
WHERE salary > 20000
GROUP BY department
HAVING AVG(salary) > 50000;
```

Q8. Difference between primary key and unique key.

A primary key uniquely identifies each row and disallows NULL. A table has one primary-key constraint but can have multiple unique constraints. A unique constraint prevents duplicate values; NULL handling varies by database.

Q9. What are relationships in DBMS?

One-to-one: one person to one passport record. One-to-many: one customer to many orders. Many-to-many: many students can enroll in many courses, usually connected by a junction table.

```
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  order_date DATE,
  FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id)
);
```

Q10. What are ACID properties?

Atomicity: all transaction steps succeed or are rolled back. Consistency: rules and constraints remain valid. Isolation: concurrent transactions are controlled to avoid improper interference. Durability: committed changes survive failures covered by the database's durability guarantees. Example: a bank transfer should not leave only the debit completed.

Q11. What is a database index?

An index is a data structure that can help the database find rows without scanning the whole table. Indexes take storage and add maintenance cost to inserts, updates, and deletes. An index does not guarantee a query will be faster; the optimizer decides whether to use it.

```
CREATE INDEX idx_employee_department
ON employees(department);
```

Q12. Clustered vs nonclustered index?

Terminology varies by database. In SQL Server, a clustered index organizes the table data, while a nonclustered index is a separate structure. In MySQL InnoDB, table data is organized around the primary key; secondary indexes store primary-key values as row references.

Part 2 — Core SQL

Q13. How do you retrieve all rows from a table?

SELECT * returns all columns. In production, listing only the needed columns is often clearer and can reduce unnecessary data retrieval.

```
SELECT * FROM employees;
```

Q14. Find employees earning more than 50,000.

Use WHERE to filter individual rows.

```
SELECT employee_id, name, salary
FROM employees
WHERE salary > 50000;
```

With the sample data, Amit, Neha, Priya, and Karan qualify.

Q15. Find distinct departments.

DISTINCT removes duplicate values or duplicate combinations of selected columns.

```
SELECT DISTINCT department
FROM employees;
```

Q16. Sort employees by salary.

ORDER BY sorts results. Add a tie-breaker to make order deterministic when salaries match.

```
SELECT name, salary
FROM employees
ORDER BY salary DESC, name ASC;
```

Q17. Find highest, lowest, and average salary.

MAX and MIN return the highest and lowest non-NULL values. AVG averages non-NULL values.

```
SELECT MAX(salary) AS highest_salary,
       MIN(salary) AS lowest_salary,
       AVG(salary) AS average_salary
FROM employees;
```

Q18. Count employees in each department.

GROUP BY creates one result group per department; COUNT(*) counts rows in each group.

```
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

Q19. Find departments with more than two employees.

Filter aggregate results using HAVING.

```
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 2;
```

Q20. Difference between COUNT(*), COUNT(column), and COUNT(DISTINCT column).

COUNT(*) counts rows. COUNT(column) counts non-NULL values in that column. COUNT(DISTINCT column) counts distinct non-NULL values.

```
SELECT COUNT(*) AS total_rows,
       COUNT(manager_id) AS employees_with_manager,
       COUNT(DISTINCT department) AS department_count
FROM employees;
```

For the sample data, the results are 6, 3, and 3.

Q21. How do you handle NULL values?

NULL means missing or unknown, not zero or an empty string. Use IS NULL or IS NOT NULL for checks. COALESCE returns the first non-NULL expression. Choose replacements carefully so missing data is not misrepresented.

```
SELECT * FROM employees
WHERE manager_id IS NULL;

SELECT name, COALESCE(manager_id, 0) AS manager_id
FROM employees;
```

Q22. What is CASE WHEN?

CASE applies conditional logic and can create categories or features, such as customer segments or salary bands.

```
SELECT name, salary,
  CASE
    WHEN salary >= 70000 THEN 'High'
    WHEN salary >= 50000 THEN 'Medium'
    ELSE 'Low'
  END AS salary_category
FROM employees;
```

Q23. UNION vs UNION ALL.

UNION combines result sets and removes duplicate rows. UNION ALL preserves duplicates and is usually preferable when deduplication is not needed. Both queries must return compatible columns in matching positions.

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers;
```

Q24. What is the logical execution order of SELECT?

A useful simplified logical order is FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT. Physical execution may differ because the optimizer can rearrange operations while preserving results. This explains why a SELECT alias generally cannot be used in WHERE.

Part 3 — SQL joins

Q25. What is a JOIN?

A JOIN combines rows from tables using a matching condition. INNER JOIN returns matches; LEFT JOIN returns every left row plus matches; RIGHT JOIN returns every right row plus matches; CROSS JOIN returns every combination; a self join joins a table to itself. MySQL does not support direct FULL OUTER JOIN syntax.

Q26. Explain INNER JOIN.

Only rows with a matching customer_id in both tables are returned.

```
SELECT c.customer_name, o.order_id, o.amount
FROM customers AS c
INNER JOIN orders AS o
  ON c.customer_id = o.customer_id;
```

Neha is absent because she has no matching order.

Q27. Return all customers, including those without orders.

LEFT JOIN preserves every row from the left table. Right-side columns are NULL when there is no match.

```
SELECT c.customer_id, c.customer_name,
```

```
       o.order_id, o.amount
FROM customers AS c
LEFT JOIN orders AS o
  ON c.customer_id = o.customer_id;
```

Q28. Find customers who never placed an order.

This is an anti-join. LEFT JOIN plus a NULL check is one solution; NOT EXISTS is another.

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- Alternative
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.customer_id
);
```

Q29. What is a self join?

A self join joins a table to itself. Aliases distinguish each role, such as employee and manager.

```
SELECT e.name AS employee_name,
       m.name AS manager_name
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id;
```

Q30. What is a CROSS JOIN?

A CROSS JOIN returns every combination of rows from two tables. Three rows crossed with four rows produce 12 rows. It can be useful for combinations but dangerous if accidental.

Q31. Common mistake with LEFT JOIN filters.

Filtering a right-side column in WHERE can remove unmatched left rows and effectively turn the result into an inner join. If you want all customers but only attach orders above 2,000, put that condition in ON.

```
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
 AND o.amount > 2000;
```

Part 4 — Subqueries and CTEs

Q32. What is a subquery?

A subquery is a query nested inside another query. This example finds employees whose salary is above the overall average.

```
SELECT name, salary
FROM employees
WHERE salary > (
  SELECT AVG(salary) FROM employees
);
```

Q33. What is a correlated subquery?

It refers to a column from the outer query. The example compares each employee's salary with the average for that employee's department. The optimizer may transform its execution.

```
SELECT e.name, e.department, e.salary
FROM employees e
WHERE e.salary > (
  SELECT AVG(e2.salary)
  FROM employees e2
  WHERE e2.department = e.department
);
```

Q34. What is a CTE?

A Common Table Expression is a named result set defined with WITH. It can make multi-step queries easier to read; it is not necessarily faster than a subquery.

```
WITH department_salary AS (
  SELECT department, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department
)
SELECT *
FROM department_salary
WHERE avg_salary > 50000;
```

Q35. CTE vs subquery.

A CTE is defined with WITH and can improve readability, especially for multi-step queries. A subquery is nested directly inside another statement and is useful for compact logic. Recursive CTEs support hierarchy and sequence problems.

Q36. What is a recursive CTE?

A recursive CTE refers to its own result and can generate sequences or traverse hierarchical data. The anchor query starts the sequence; the recursive query produces subsequent rows until the condition stops.

```
WITH RECURSIVE numbers AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM numbers WHERE n < 5
)
SELECT n FROM numbers;
```

Part 5 — Window functions

Q37. ROW_NUMBER vs RANK vs DENSE_RANK.

ROW_NUMBER assigns a unique sequence number. RANK gives ties the same rank and skips ranks afterward. DENSE_RANK gives ties the same rank without gaps. Add a tie-breaker to ROW_NUMBER when deterministic ordering is needed.

```
SELECT name, salary,
  ROW_NUMBER() OVER (ORDER BY salary DESC, employee_id) AS row_num,
  RANK() OVER (ORDER BY salary DESC) AS salary_rank,
  DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM employees;
```

Q38. Find the second-highest distinct salary.

MAX with a less-than condition finds the maximum salary below the highest. DENSE_RANK is a good alternative. The result is NULL if there is no second distinct salary.

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Window-function alternative
WITH ranked AS (
  SELECT salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
)
SELECT MAX(salary)
FROM ranked
WHERE rnk = 2;
```

Q39. Find the highest-paid employee in each department.

PARTITION BY ranks each department separately. DENSE_RANK returns all employees tied for the highest salary.

```
WITH ranked AS (
  SELECT employee_id, name, department, salary,
    DENSE_RANK() OVER (
      PARTITION BY department ORDER BY salary DESC
    ) AS rnk
  FROM employees
)
```

```
SELECT employee_id, name, department, salary
FROM ranked WHERE rnk = 1;
```

If exactly one row per department is required, use ROW_NUMBER with a deterministic tie-breaker.

Q40. What is PARTITION BY?

PARTITION BY divides rows into groups for a window calculation without collapsing the individual rows.

```
SELECT name, department, salary,
  AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM employees;
```

Q41. GROUP BY vs window function.

GROUP BY usually returns one row per group. A window function retains individual rows while adding group-level metrics such as department averages.

Q42. Calculate a running total.

SUM with an ordered window frame adds the current row to all preceding rows. If ordering values can tie, add a unique ordering column.

```
SELECT sale_date, amount,
  SUM(amount) OVER (
    ORDER BY sale_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total
FROM daily_sales;
```

Q43. Calculate month-over-month revenue growth.

Aggregate revenue by month, use LAG to get the previous month's revenue, then calculate percentage change. A complete calendar is needed if months with no orders must appear.

```
WITH monthly_sales AS (
  SELECT DATE_FORMAT(order_date, '%Y-%m') AS sales_month,
         SUM(amount) AS revenue
  FROM orders
  GROUP BY DATE_FORMAT(order_date, '%Y-%m')
),
comparison AS (
  SELECT sales_month, revenue,
    LAG(revenue) OVER (ORDER BY sales_month) AS previous_revenue
  FROM monthly_sales
)
SELECT sales_month, revenue, previous_revenue,
  ROUND(100.0 * (revenue - previous_revenue)
    / NULLIF(previous_revenue, 0), 2) AS growth_percentage
FROM comparison
ORDER BY sales_month;
```

Q44. What are LAG and LEAD?

LAG accesses a previous row's value; LEAD accesses a subsequent row's value. They are useful for trend analysis, comparisons, and time-series features.

```
SELECT sale_date, amount,
  LAG(amount) OVER (ORDER BY sale_date) AS previous_amount,
  LEAD(amount) OVER (ORDER BY sale_date) AS next_amount
FROM daily_sales;
```

Q45. Calculate a moving average.

This calculates the average for the current row and six previous rows. It is a seven-day average only if there is exactly one row per calendar day with no missing dates.

```
SELECT sale_date, amount,
  AVG(amount) OVER (
    ORDER BY sale_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS seven_row_avg
FROM daily_sales;
```

Part 6 — Common SQL coding problems

Q46. Find duplicate email addresses.

Group by email and retain groups with more than one occurrence. This version excludes NULL emails; normalize case and whitespace if business rules require it.

```
SELECT email, COUNT(*) AS occurrences
FROM users
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

Q47. Identify duplicate records but keep one.

Use ROW_NUMBER to rank records with the same email and keep the smallest user_id. First inspect the rows marked for removal and back up data before deleting.

```
WITH ranked AS (
  SELECT user_id, email,
    ROW_NUMBER() OVER (
      PARTITION BY email ORDER BY user_id
    ) AS rn
  FROM users
  WHERE email IS NOT NULL
)
SELECT * FROM ranked WHERE rn > 1;
```

Q48. Find employees who earn more than their manager.

Join employees to the same table using manager_id, then compare salaries.

```
SELECT e.name AS employee_name, e.salary AS employee_salary,
       m.name AS manager_name, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

Q49. Find the top three highest-paid employees in each department.

ROW_NUMBER returns up to three employees per department, breaking salary ties by employee_id. Use DENSE_RANK if you want all employees in the top three distinct salary levels.

```
WITH ranked AS (
  SELECT employee_id, name, department, salary,
    ROW_NUMBER() OVER (
      PARTITION BY department
      ORDER BY salary DESC, employee_id
    ) AS rn
  FROM employees
)
SELECT * FROM ranked
WHERE rn <= 3
ORDER BY department, rn;
```

Q50. Find customers whose total spending is above 4,000.

Join customers to orders, aggregate spending per customer, then filter the groups with HAVING.

```
SELECT c.customer_id, c.customer_name,
       SUM(o.amount) AS total_spending
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING SUM(o.amount) > 4000;
```

For the sample data, Aman has 4,500 and Rahul has 5,000 in spending.

Q51. Calculate each department's percentage of total salary.

First calculate total salary per department, then divide each department total by the overall total using a window sum.

```
WITH department_totals AS (
  SELECT department, SUM(salary) AS total_salary
  FROM employees GROUP BY department
)
SELECT department, total_salary,
```

```
  ROUND(100.0 * total_salary /
    SUM(total_salary) OVER (), 2) AS salary_percentage
FROM department_totals;
```

Q52. Find products that have never been sold.

Use NOT EXISTS to find products without a matching row in order_items.

```
SELECT p.product_id, p.product_name
FROM products p
WHERE NOT EXISTS (
  SELECT 1 FROM order_items oi
  WHERE oi.product_id = p.product_id
);
```

Q53. Find dates that follow the previous sales date by one day.

LAG gets the previous recorded date. DATEDIFF checks whether the current date is one day later. This finds adjacent dates, not complete streaks by itself.

```
WITH dated AS (
  SELECT sale_date,
    LAG(sale_date) OVER (ORDER BY sale_date) AS previous_date
  FROM daily_sales
)
SELECT sale_date, previous_date
FROM dated
WHERE DATEDIFF(sale_date, previous_date) = 1;
```

Q54. Find each customer's first order.

ROW_NUMBER ranks orders per customer by date and order ID, returning one earliest order per customer who has ordered.

```
WITH ranked AS (
  SELECT order_id, customer_id, order_date, amount,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id ORDER BY order_date, order_id
    ) AS rn
  FROM orders
)
SELECT order_id, customer_id, order_date, amount
FROM ranked WHERE rn = 1;
```

Q55. Find each customer's most recent purchase date.

MAX returns the latest order date. LEFT JOIN keeps customers who have never ordered; their last_purchase_date is NULL.

```
SELECT c.customer_id, c.customer_name,
       MAX(o.order_date) AS last_purchase_date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name;
```

Part 7 — SQL for Data Science

Q56. Calculate monthly revenue.

Group orders by year and month and sum the amount. In real reporting, define how cancellations, refunds, discounts, and taxes are handled.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       SUM(amount) AS monthly_revenue
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

Q57. Calculate average order value (AOV).

AOV is commonly total order revenue divided by the number of orders. This assumes one row per order; aggregate order-item rows to order level first if necessary.

```
SELECT SUM(amount) / NULLIF(COUNT(DISTINCT order_id), 0)
       AS average_order_value
FROM orders;
```

Q58. Find repeat customers.

A repeat customer has more than one qualifying order in the specified period. Define the time period and whether cancelled orders count.

```
SELECT customer_id, COUNT(DISTINCT order_id) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(DISTINCT order_id) > 1;
```

Repeat-customer rate = customers with 2+ orders divided by customers with 1+ orders, multiplied by 100, if that is the agreed business definition.

Q59. How do you calculate customer retention?

Cohort retention groups customers by their first purchase month and counts how many return in later months. Divide each month's active customers by the original cohort size to get a percentage.

```
WITH customer_months AS (
  SELECT DISTINCT customer_id,
    CAST(DATE_FORMAT(order_date, '%Y-%m-01') AS DATE) AS activity_month
  FROM orders
),
cohorts AS (
  SELECT customer_id, MIN(activity_month) AS cohort_month
  FROM customer_months GROUP BY customer_id
)
SELECT c.cohort_month, a.activity_month,
       COUNT(DISTINCT a.customer_id) AS retained_customers
FROM cohorts c
JOIN customer_months a ON c.customer_id = a.customer_id
GROUP BY c.cohort_month, a.activity_month
ORDER BY c.cohort_month, a.activity_month;
```

Q60. How would you clean a dataset using SQL?

Check missing values, duplicates, types and formats, invalid values, and referential integrity. Confirm business rules before removing records: a negative amount could represent a refund.

```
SELECT * FROM orders WHERE customer_id IS NULL;

SELECT order_id, COUNT(*) AS occurrences
FROM orders
GROUP BY order_id
HAVING COUNT(*) > 1;

SELECT * FROM orders WHERE amount < 0;
```

Q61. How would you create machine-learning features in SQL?

For churn prediction, features might include order count, total spending, average order value, and days since the last purchase. Use a defined prediction cutoff date and only information available by that time.

```
SELECT c.customer_id,
       COUNT(o.order_id) AS total_orders,
       COALESCE(SUM(o.amount), 0) AS total_spending,
       COALESCE(AVG(o.amount), 0) AS avg_order_value,
       DATEDIFF(DATE('2026-10-01'), MAX(o.order_date))
         AS days_since_last_order
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id;
```

The date is illustrative. Customers without orders have NULL days_since_last_order; decide how to represent that and consider adding a no-purchase indicator.

Q62. How would you investigate a sudden revenue drop?

Validate data completeness first. Compare periods, segment by product, region, channel, and customer type; inspect order volume, average order value, cancellations, refunds, and pipeline changes. Report evidence-based likely causes.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       COUNT(DISTINCT order_id) AS orders,
       SUM(amount) AS revenue,
       AVG(amount) AS avg_order_value
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

Q63. How would you identify sales outliers in SQL?

One option is to flag values more than three population standard deviations from the mean. Sales can be skewed, so also consider percentiles, interquartile ranges, robust statistics, and business rules.

```
WITH stats AS (
  SELECT AVG(amount) AS avg_amount,
         STDDEV_POP(amount) AS std_amount
  FROM orders
)
SELECT o.*
FROM orders o CROSS JOIN stats s
WHERE s.std_amount > 0
  AND ABS(o.amount - s.avg_amount) > 3 * s.std_amount;
```

Q64. How would you combine SQL and Python in a Data Science project?

Use SQL to retrieve, join, aggregate, and validate data near its source. Use Python/pandas for exploratory analysis and preparation, then suitable libraries such as scikit-learn for modeling and evaluation. Validate outputs, document transformations, and automate repeatable workflows.

Part 8 — Advanced SQL and performance

Q65. What is query optimization?

Query optimization aims to reduce execution cost while preserving results. Select needed columns, use suitable indexes, check joins and data types, avoid unnecessary repeated work, and inspect the execution plan.

```
EXPLAIN
SELECT employee_id, name
FROM employees
WHERE department = 'IT';
```

Q66. Why can a query be slow even when an index exists?

The query may return many rows, the index may be unselective or poorly ordered, a function on the indexed column may prevent an efficient lookup, or joins/sorts may dominate. Check EXPLAIN and measure representative workloads.

```
-- Often easier to optimize with an index on salary
SELECT employee_id FROM employees WHERE salary >= 50000;

-- Applying arithmetic to the column may make index use harder
SELECT employee_id FROM employees WHERE salary + 1000 >= 51000;
```

These two predicates are equivalent in this example; the second can be rewritten as salary >= 50000.

Q67. What is a composite index?

A composite index includes multiple columns. An index on (customer_id, order_date) can support searches by customer_id and by customer_id plus date. It may not efficiently support a search by order_date alone because customer_id is the leading column.

```
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);
```

Q68. What is a transaction?

A transaction is a logical unit of work that can be committed or rolled back. In a transfer, debit and credit should be handled together. Validate constraints and concurrency; a transaction alone does not guarantee every business rule.

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 1000
WHERE account_id = 1;
UPDATE accounts SET balance = balance + 1000
WHERE account_id = 2;
COMMIT;
```

Q69. View vs materialized view.

A view stores a query definition and presents its results when queried. A materialized view stores query results and needs refresh or maintenance. MySQL supports regular views but not native materialized views in the same way as some other systems; tables plus refresh logic can be used.

Q70. What are database locks?

Locks coordinate concurrent access. Shared locks allow compatible readers; exclusive locks restrict conflicting access while changes are made. Row-level and table-level locks differ in scope. Exact behavior depends on the database engine and isolation level.

Q71. What is a deadlock?

A deadlock occurs when transactions wait on one another in a cycle. A database such as InnoDB detects the deadlock and typically rolls one transaction back. Reduce risk by accessing rows in a consistent order, keeping transactions short, and safely retrying failed transactions.

Q72. Explain transaction isolation levels.

Read Uncommitted may allow dirty reads; Read Committed prevents dirty reads; Repeatable Read provides stronger consistency for repeated reads; Serializable provides the strongest standard isolation but may restrict concurrency. Exact behavior varies. MySQL InnoDB defaults to Repeatable Read.

Part 9 — Business scenarios and tricky questions

Q73. Find the percentage of customers who have placed at least one order.

This counts all customers in the denominator, including those who never ordered. The definition of the denominator should match the business question.

```
WITH customer_order_counts AS (
  SELECT c.customer_id, COUNT(o.order_id) AS order_count
  FROM customers c
  LEFT JOIN orders o ON c.customer_id = o.customer_id
  GROUP BY c.customer_id
)
SELECT ROUND(
  100.0 * SUM(CASE WHEN order_count > 0 THEN 1 ELSE 0 END)
  / NULLIF(COUNT(*), 0), 2
) AS customer_conversion_percentage
FROM customer_order_counts;
```

For the sample data, 3 of 4 customers ordered, so the result is 75%.

Q74. Find customers spending above the average customer spending.

Aggregate spending per customer first, including non-buyers as zero, then compare each customer with the average across all customers.

```
WITH customer_spending AS (
  SELECT c.customer_id, c.customer_name,
         COALESCE(SUM(o.amount), 0) AS total_spending
  FROM customers c
  LEFT JOIN orders o ON c.customer_id = o.customer_id
  GROUP BY c.customer_id, c.customer_name
)
SELECT *
FROM customer_spending
WHERE total_spending > (SELECT AVG(total_spending)
                        FROM customer_spending);
```

Q75. How do you find missing dates in a time series?

Define the expected date range and frequency, generate a calendar table or recursive date sequence, and LEFT JOIN observed dates to it. Missing dates matter before calculating daily rolling averages or forecasting features.

Q76. How do you prevent data leakage when preparing training data?

Define a prediction timestamp and use only features available at or before it. Exclude future outcomes, calculate historical aggregates using appropriate windows, split data to match deployment conditions, and verify that validation/test information does not influence feature generation. For time-dependent data, a chronological split is often appropriate.

Part 10 — Rapid-fire questions

Question	Simple answer
What is a constraint?	A rule such as NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, or CHECK.
What is a schema?	A logical organization of database objects.
What is a stored procedure?	A named routine stored in the database that executes SQL statements.
What is a trigger?	A database object that runs automatically for specified events.
What is a scalar function?	A function returning one value, such as UPPER() or ROUND().
What is the NOT IN NULL trap?	A NULL in the subquery can make NOT IN results unexpected; NOT EXISTS is often safer.
What is a surrogate key?	An artificial identifier, such as an auto-incrementing integer.
What is a natural key?	An identifier derived from real-world business data.
What is referential integrity?	Ensuring relationships between tables remain valid.
What is denormalization?	Intentionally combining or duplicating data for particular access or analytical needs.
What is a data warehouse?	A system consolidating data for analysis and reporting.
What is a fact table?	A table recording measurable events such as sales or transactions.
What is a dimension table?	A table describing entities such as customers, products, or dates.
What is a star schema?	A central fact table connected to dimension tables.
What is an execution plan?	A representation of how the database plans to execute a query.

Part 11 — How to answer like a 2-year candidate

Do more than write a query: clarify requirements, explain the logic, mention edge cases, and discuss trade-offs.

Example: If asked for the second-highest salary, clarify whether the interviewer wants the second-highest distinct salary or the employee in the second row of salary order. For the second-highest distinct salary, DENSE_RANK or a MAX subquery is suitable. Also consider a table with fewer than two distinct salaries.

Evaluation checklist:

Area	What the interviewer checks
Query correctness	Does the query return the intended records?
Edge cases	Can you handle NULLs, duplicates, ties, and empty tables?
Business understanding	Can you define revenue, retention, and other metrics correctly?
Performance	Can you reason about indexes, joins, and execution plans?
Data Science awareness	Do you understand feature engineering, leakage, and time-based validation?
Communication	Can you explain the logic and trade-offs clearly?

Part 12 — 14-day preparation plan

Days	Focus	Practice
1–3	DBMS and core SQL	Keys, normalization, constraints, SELECT, WHERE, GROUP BY, HAVING, aggregates, NULL.
4–6	Joins and subqueries	Customer/order joins, missing records, self joins, correlated subqueries, CTEs.
7–9	Window functions	ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, running totals, moving averages.
10–12	Data Science cases	Retention, RFM features, revenue analysis, data cleaning, leakage prevention.
13–14	Advanced SQL and mocks	Indexes, EXPLAIN, transactions, timed coding, explaining queries aloud.

Recommended practice websites

HackerRank SQL — https://www.hackerrank.com/domains/sql

LeetCode Database — https://leetcode.com/problemset/database/

SQLBolt — https://sqlbolt.com/

MySQL documentation — https://dev.mysql.com/doc/

Final recommendation: Prioritize joins, GROUP BY/HAVING, subqueries and CTEs, window functions, top-N problems, data cleaning, customer analytics, query optimization, and leakage prevention. A Data Scientist interview may also test Python, pandas, statistics, machine learning, and project explanations.